



DSN x BCT Hackathon 3.0 — LLM Agent Challenge

Team: HOKM | **Tasks:** A (User Modeling) + B (Recommendation)

What We're Building

Most recommendation systems treat users as static profiles — fixed preferences locked in a database. **PulseAgent** treats every user as a dynamic, context-sensitive agent whose behaviour, language, and choices shift depending on history, context, and intent.

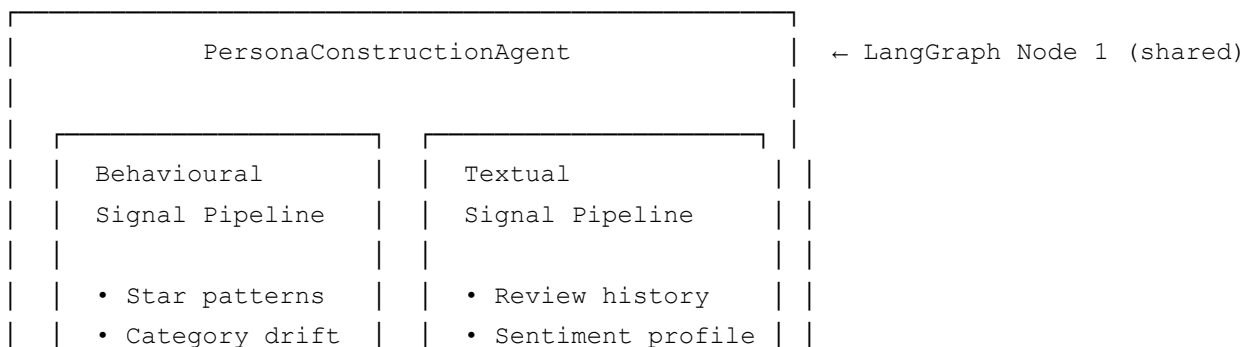
We build two agents:

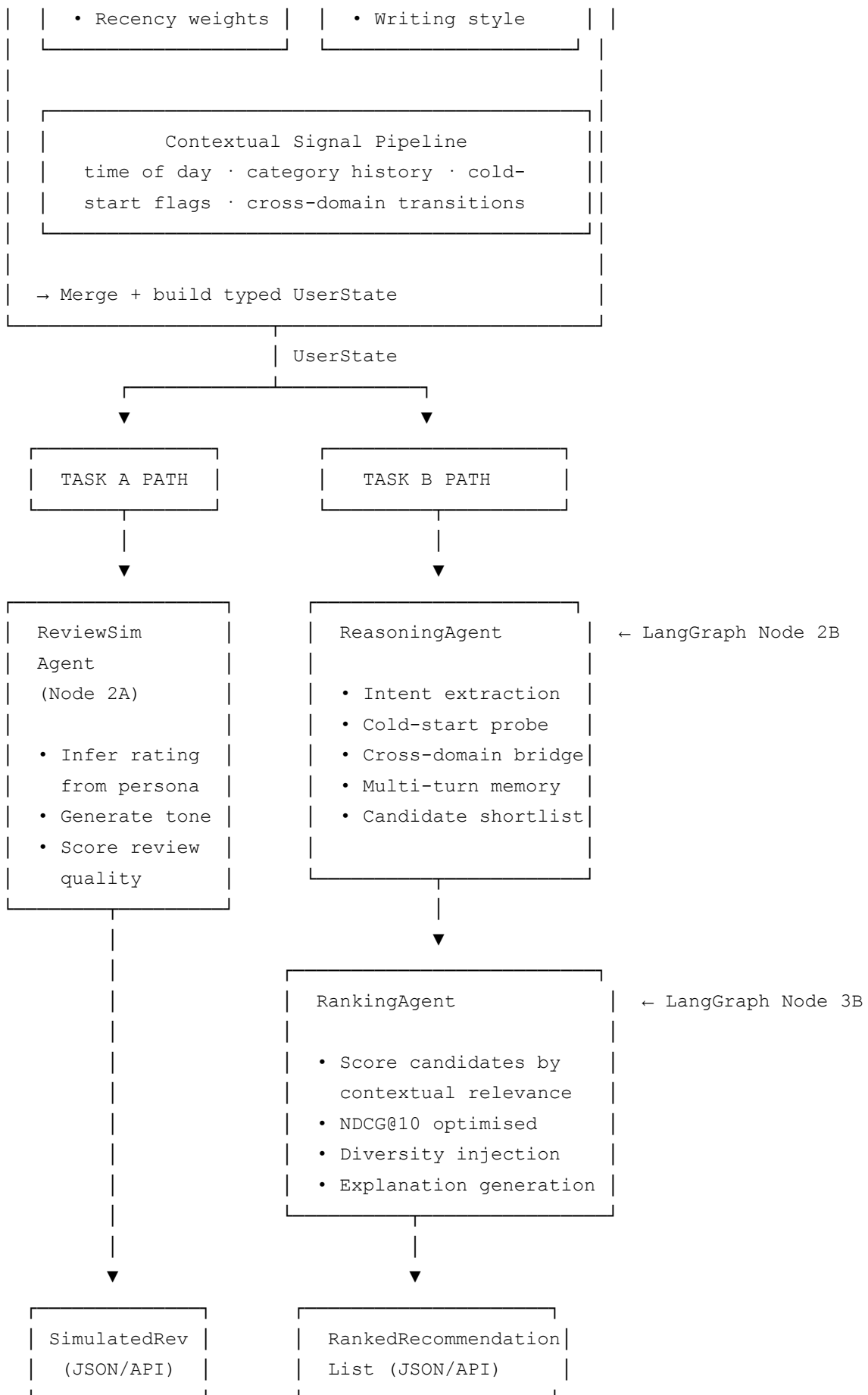
- **Task A — The Persona Engine:** Given a user's history and an unseen item, simulate how that user would review it — their star rating, their tone, their reasoning.
- **Task B — The Recommendation Engine:** Given a user persona, reason through what they actually want right now — handling cold starts, cross-domain leaps, and multi-turn conversations before making a ranked recommendation.

Both agents share the same underlying user state representation. One understands people. The other uses that understanding to serve them.

System Architecture

UserPersona + ItemDetails / UserPersona





LangGraph State Flow

```
class AgentState(TypedDict):
    # Input
    user_persona: UserPersona
    item_details: Optional[ItemDetails]          # Task A only
    conversation_history: List[Message]          # Task B multi-turn

    # Constructed in Node 1
    user_state: UserState                        # typed behavioural + textual profil

    # Task A outputs (Node 2A)
    simulated_rating: Optional[float]
    simulated_review: Optional[str]
    review_quality_score: Optional[float]
    reasoning_trace: List[str]

    # Task B outputs (Nodes 2B + 3B)
    inferred_intent: Optional[str]
    candidate_items: List[CandidateItem]
    ranked_recommendations: List[RankedItem]
    explanation: Optional[str]
```

Team Responsibilities

4 members. Clean split. Each owns a vertical end-to-end — schema through tests.

Member 1 — Henry (Team Lead + Task A: ReviewSim Agent)

Owns: Architecture, LangGraph graph orchestration, PersonaConstructionAgent (Node 1), ReviewSimAgent (Node 2A), Docker + CI

Deliverables:

- `src/graph.py` — full LangGraph pipeline wiring for both tasks
- `src/agents/persona_agent.py` — Node 1: 3-pipeline persona construction
- `src/agents/review_agent.py` — Node 2A: rating inference + review generation
- `src/schemas/models.py` — all Pydantic types (shared ground truth for the team)

- `Dockerfile` + `docker-compose.yml`
- Final solution paper (lead author)
- README

Key decisions owned:

- State schema design
 - LangGraph routing logic between Task A and Task B paths
 - Tone inference strategy from user history
-

Member 2 — [Name] (Task B: Reasoning + Ranking Agents)

Owns: ReasoningAgent (Node 2B), RankingAgent (Node 3B), cold-start and cross-domain logic

Deliverables:

- `src/agents/reasoning_agent.py` — intent extraction, cold-start probe, multi-turn memory
- `src/agents/ranking_agent.py` — candidate scoring, NDCG@10-optimised ranking, diversity injection
- `src/tools/retrieval_tool.py` — item candidate retrieval from dataset index
- Cold-start scenario handler (new user / new item coverage)
- Cross-domain bridge logic (e.g. user who reviews food → recommend books)

Key decisions owned:

- Retrieval strategy (embedding similarity vs BM25 vs hybrid)
 - Ranking objective formulation
 - Multi-turn conversation state management
-

Member 3 — [Name] (Data Pipeline + Evaluation)

Owns: Dataset preprocessing, feature extraction, evaluation harness, all metrics

Deliverables:

- `src/data/yelp_loader.py` — Yelp dataset ingestion + user-item graph construction
- `src/data/amazon_loader.py` — Amazon Reviews ingestion
- `src/data/goodreads_loader.py` — Goodreads ingestion
- `src/data/preprocessor.py` — unified schema normalisation across all three datasets
- `src/evaluation/metrics.py` — ROUGE-L, BERTScore, RMSE, NDCG@10, Hit Rate, cold-start coverage
- `src/evaluation/runner.py` — automated eval runner against held-out test set
- Ablation study scripts (used in solution paper Section 4)

Key decisions owned:

- Train/test split strategy per dataset
- Human eval rubric design (behavioural fidelity, contextual relevance)

Member 4 — [Name] (API Layer + Frontend + Solution Paper)

Owns: FastAPI endpoints for both tasks, containerised web UI, solution paper write-up

Deliverables:

- `src/api/main.py` — FastAPI app with `/simulate-review` (Task A) and `/recommend` (Task B)
- `src/api/schemas.py` — request/response models for API layer
- `frontend/app.py` — Streamlit demo UI
- `tests/` — full test suite (target: 80+ tests across all layers)
- Solution paper (primary writer, working from architecture + results provided by Members 1–3)

Key decisions owned:

- API contract design (inputs, outputs, error handling)
 - Demo UX flow for judges
 - Paper structure, ablation narrative, and writing clarity
-

Project Structure

```
pulse-agent/
|
├─ main.py                # CLI entrypoint (both tasks)
├─ requirements.txt
├─ Dockerfile
├─ docker-compose.yml
├─ pytest.ini
├─ README.md
|
├─ src/
|   |
|   ├─ schemas/
|   |   └─ models.py      # All Pydantic types – owned by Member 1
|   |                       # UserPersona, ItemDetails, UserState,
|   |                       # SimulatedReview, CandidateItem,
|   |                       # RankedItem, AgentState
|   |
|   └─ agents/
|       ├─ persona_agent.py # Node 1: 3-pipeline persona construction (M
|       ├─ review_agent.py  # Node 2A: rating + review simulation (Membe
|       ├─ reasoning_agent.py # Node 2B: intent + cold-start + multi-turn
|       ├─ ranking_agent.py  # Node 3B: candidate scoring + ranking (Memb
|       └─ graph.py          # LangGraph graph construction (Member 1)
|
|   └─ tools/
|       ├─ retrieval_tool.py # Item candidate retrieval (Member 2)
|       └─ embedding_tool.py # Sentence embedding utility (Member 2)
|
|   └─ data/
|       ├─ yelp_loader.py    # Yelp dataset ingestion (Member 3)
|       ├─ amazon_loader.py  # Amazon Reviews ingestion (Member 3)
|       ├─ goodreads_loader.py # Goodreads ingestion (Member 3)
|       └─ preprocessor.py    # Unified schema normalisation (Member 3)
|
|   └─ evaluation/
|       ├─ metrics.py        # ROUGE-L, BERTScore, RMSE, NDCG@10 (Member
|       └─ runner.py         # Automated eval runner (Member 3)
|
|   └─ api/
|       ├─ main.py           # FastAPI app (Member 4)
|       └─ schemas.py        # API request/response models (Member 4)
|
|   └─ utils/
|       └─ formatter.py      # Terminal (Rich), JSON, Markdown renderers
```

```
|
|— frontend/
|   └─ app.py                                # Streamlit demo UI (Member 4)
|
|— tests/
|   └─ test_schemas.py
|   └─ test_persona_agent.py
|   └─ test_review_agent.py
|   └─ test_reasoning_agent.py
|   └─ test_ranking_agent.py
|   └─ test_data_loaders.py
|   └─ test_metrics.py
|   └─ test_api.py
|   └─ test_integration.py
|
└─ paper/
    └─ pulse_agent_solution_paper.pdf
```

API Contracts

Task A — `POST /simulate-review`

Request:

```
{
  "user_persona": {
    "user_id": "u_001",
    "review_history": [
      { "item_id": "i_042", "category": "Food", "rating": 4.0, "text": "Great spo
    ],
    "avg_rating": 3.8,
    "preferred_categories": ["Food", "Nightlife"],
    "tone_profile": "expressive"
  },
  "item_details": {
    "item_id": "i_199",
    "name": "The Grill House",
    "category": "Food",
    "attributes": { "cuisine": "American", "price_range": "$$" }
  }
}
```

Response:

```
{
  "simulated_rating": 4.5,
  "simulated_review": "Honestly one of the better spots I've been to in a while.",
  "confidence": 0.87,
  "reasoning_trace": [
    "User consistently rates casual dining 0.5 above their average...",
    "Tone profile: expressive → short, punchy sentences with strong opinion markers"
  ]
}
```

Task B — POST /recommend

Request:

```
{
  "user_persona": {
    "user_id": "u_042",
    "review_history": [...],
    "conversation_history": [
      { "role": "user", "content": "I want something chill for this weekend, not too loud." }
    ]
  }
}
```

Response:

```
{
  "recommendations": [
    {
      "rank": 1,
      "item_id": "i_305",
      "name": "The Rooftop Lounge",
      "category": "Nightlife",
      "predicted_rating": 4.3,
      "explanation": "Matches your preference for casual, mid-range spots and weekend vibes.",
      "ndcg_score": 0.91
    }
  ],
  "inferred_intent": "casual weekend outing, budget-conscious",
  "cold_start": false,
  "reasoning_trace": [...]
}
```

Key Design Decisions

Single shared `UserState`. Both tasks derive from the same typed persona representation built in Node 1. Task A and Task B are always working from the same understanding of the user — no drift between the reviewer and the recommender.

Three concurrent persona pipelines. Behavioural signals, textual signals, and contextual signals run as concurrent `asyncio` tasks. Each fails independently — a missing review history doesn't crash the textual pipeline.

Cold-start as a first-class scenario. The `ReasoningAgent` explicitly detects cold-start conditions (new user, sparse history, new item category) and switches strategy — falling back to demographic priors, cross-domain transfer, or explicit clarifying questions in multi-turn mode.

Structured outputs throughout. Every object flowing through the `LangGraph` graph is typed with `Pydantic`. The LLM returns JSON; the parser handles malformed responses gracefully without crashing the pipeline.

Auditable reasoning traces. Every simulated review and every recommendation carries a `reasoning_trace` — the model's chain-of-thought explaining why this rating, why this tone, why this item.

Scoring Alignment

Rubric Item	Our Coverage
Review Text Quality (ROUGE / BERTScore)	<code>evaluation/metrics.py</code> — automated on held-out set
Rating Accuracy (RMSE)	Measured against ground-truth ratings in test split
Behavioural Fidelity (human eval)	Reasoning traces support human evaluators
Ranking Quality (NDCG@10 / Hit Rate)	<code>ranking_agent.py</code> optimised for NDCG
Cold-Start & Cross-Domain	Explicit handler in <code>reasoning_agent.py</code>
Contextual Relevance (human eval)	Explanation field in every recommendation
Solution Paper	4–8 pages, Member 4 lead, ablation studies from

	Member 3
Code Reproducibility	Docker + README + modular structure

Milestones

Date	Milestone
Week 1	Architecture locked, repo initialised, schema models agreed
Week 2	Data loaders complete, PersonaConstructionAgent (Node 1) working
Week 3	ReviewSimAgent (Task A) + ReasoningAgent (Task B) end-to-end
Week 4	RankingAgent done, evaluation harness running
May 23	All deliverables ready, paper first draft done
May 24	Final submission

Team HOKM

Member	Role	Key Ownership
Henry	Team Lead + Task A Agent	Architecture, LangGraph, ReviewSimAgent, Docker
[Name 2]	Task B: Reasoning + Ranking	ReasoningAgent, RankingAgent, cold-start, retrieval
[Name 3]	Data + Evaluation	Dataset loaders, metrics, ablation studies
[Name 4]	API + Frontend + Paper	FastAPI, Streamlit UI, tests, solution paper

